



SmartLib Pro

Automated Library Management System

System Guide & Technical Reference

Version 1.0 · July 2026

Table of Contents

3	1.	System Overview	
4	2.	Architecture & Tech Stack	
5	3.	Database Schema	
6	4.	API Endpoints Reference	
7	5.	User Roles & Permissions	
8	6.	Feature Walkthrough	
11	7.	Authentication Flow	
12	8.	Demo Accounts & Quick Start	
13	9.	Deployment & Configuration	

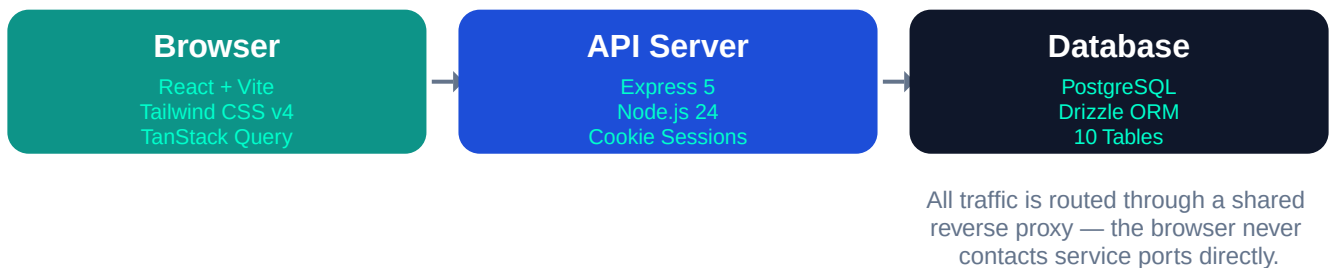
1. System Overview

SmartLib Pro is a full-stack automated library management system built for universities, schools, and public libraries. It replaces manual record-keeping with a real-time web application that handles every aspect of library operations — from book cataloguing to fine collection — through a clean, role-aware interface.

What the System Provides

- Centralised catalogue of books with author, category, publisher, and availability data
- Member management with role-based access (Admin · Librarian · Student)
- Full borrowing lifecycle: issue, return, renew, and automatic overdue detection
- Hold / reservation queue for unavailable books
- Automatic fine calculation for overdue items with a pay-off workflow
- Per-user notification feed (overdue alerts, due reminders, reservation ready, etc.)
- Admin analytics dashboard: live stats, borrowing trends, category distribution
- Immutable activity log for compliance and audit purposes
- System setup screens to manage authors, categories, and publishers

High-Level System Diagram



Key Design Choices

- Contract-first API: OpenAPI spec !' Orval generates typed React Query hooks + Zod validators
- Cookie-based sessions (httpOnly) — no JWT tokens exposed to JavaScript
- Monorepo (pnpm workspaces) with shared lib packages for DB schema, API spec, and generated clients
- Role enforcement at both route level (Express middleware) and UI level (React protected routes)

2. Architecture & Tech Stack

Monorepo Structure

The project uses pnpm workspaces with a clear separation of concerns:

```
artifacts-monorepo/
% % % artifacts/
%   % % % api-server/      Express 5 backend (port 8080 !' proxied at /api)
%   % % % smartlib-pro/    React + Vite frontend (port 23393 !' proxied at /)
% % % lib/
%   % % % api-spec/        openapi.yaml + Orval codegen config
%   % % % api-client-react/ Generated React Query hooks + custom fetch
%   % % % api-zod/         Generated Zod request/response validators
%   % % % db/              Drizzle ORM schema + migration config
% % % scripts/             Utility scripts (e.g. PDF generation)
% % % pnpm-workspace.yaml
% % % tsconfig.base.json
```

Technology Stack

Layer	Technology	Purpose
Frontend	React 18 + Vite 7	Component rendering, HMR dev server
UI Library	shadcn/ui + Tailwind v4	Pre-built accessible components + utility CSS
Routing	Wouter	Lightweight client-side routing
Data Fetching	TanStack Query v5	Server state, caching, background refetch
API Client	Orval (generated)	Typed hooks from OpenAPI spec
Backend	Express 5 + Node.js 24	REST API server
Database	PostgreSQL + Drizzle ORM	Relational data, type-safe queries
Validation	Zod v4 (generated)	Runtime request/response validation
Auth	PBKDF2 + httpOnly cookie	Secure password hashing, session cookies
Build	esbuild (CJS bundle)	Fast backend bundling
Typecheck	TypeScript 5.9 strict	End-to-end type safety

Codegen Workflow

When the API contract changes, a single command regenerates all client code:

```
pnpm --filter @workspace/api-spec run codegen
```

- Reads lib/api-spec/openapi.yaml
- Writes typed React Query hooks to lib/api-client-react/src/generated/
- Writes Zod schemas to lib/api-zod/src/generated/
- Run pnpm run typecheck:libs after to rebuild declaration files

3. Database Schema

All tables live in the default PostgreSQL schema. Drizzle ORM provides type-safe queries. Run `pnpm --filter @workspace/db run push` to sync schema changes to the database.

users

Stores all system users — admins, librarians, and students.

<code>id</code>	<code>serial PK</code>	Auto-incrementing primary key
<code>email</code>	<code>text UNIQUE</code>	Login identifier
<code>password_hash</code>	<code>text</code>	PBKDF2 hash (salt:hash format)
<code>name</code>	<code>text</code>	Display name
<code>role</code>	<code>enum</code>	admin librarian student
<code>phone / student_id</code>	<code>text?</code>	Optional contact / institutional ID
<code>is_active</code>	<code>boolean</code>	Soft-disable accounts without deletion
<code>created_at</code>	<code>timestamp</code>	Account creation time

books

Core inventory. Each row is a title; copies count tracked in place.

<code>id</code>	<code>serial PK</code>	
<code>title</code>	<code>text</code>	Book identity
<code>isbn</code>	<code>text</code>	Book identity
<code>author_id / category_id / publisher_id</code>	<code>FK</code>	Relations to lookup tables
<code>total_copies / available_copies</code>	<code>integer</code>	Inventory counters
<code>shelf_location</code>	<code>text?</code>	Physical location code (e.g. A-01)
<code>published_year / language / description</code>	<code>various</code>	Metadata fields

borrow_records

Tracks every loan. Status transitions: active !' returned | overdue.

<code>id</code>	<code>serial PK</code>	
<code>user_id</code>	<code>FK</code>	Who borrowed
<code>book_id</code>	<code>FK</code>	which book
<code>borrowed_at / due_date / returned_at</code>	<code>timestamp</code>	Lifecycle dates
<code>status</code>	<code>enum</code>	active overdue returned

`renew_count`
`integer` Max 2
renewals per loan

reservations

Queue for unavailable books.

`status`
`enum` pending |
fulfilled | cancelled | expired
`expires_at`
`timestamp?` Auto-expiry
date for pending holds

fines

One row per overdue loan. Amount in USD.

`amount`
`numeric` Calculated
at issue time (\$0.50/day default)
`status`
`enum` unpaid | paid
`paid_at`
`timestamp?` Set when
librarian marks paid

notifications

Per-user notification feed.

`type`
`enum`
overdue_alert | fine_issued | reservation_ready
| due_reminder | general
`is_read`
`boolean` Dismissed
via UI

activity_logs

Immutable audit trail. Never deleted.

`action`
`text` e.g.
book_borrowed, user_created, fine_paid
`entity_type` /
`entity_id` `text` /
`int` What was acted upon
`details`
`text?` Human-
readable description

4. API Endpoints Reference

All endpoints are prefixed with /api. The base URL in development is <http://localhost:80/api> (via the shared proxy). Authentication is enforced on every endpoint except POST /auth/login.

Authentication

POST	/auth/login	[Public]	Login with email + password. Sets session cookie.
POST	/auth/logout	[Any]	Destroys session. Clears cookie.
GET	/auth/me	[Any]	Returns current user profile.

Books

GET	/books	[Any]	List books with search, category, pagination
POST	/books	[Admin/Librarian]	Create a new book record
GET	/books/:id	[Any]	Get a single book by ID
PUT	/books/:id	[Admin/Librarian]	Update book details
DELETE	/books/:id	[Admin]	Remove book from catalogue

Catalog (Authors / Categories / Publishers)

GET/POST	/authors	[Mixed]	List or create authors
PUT/DELETE	/authors/:id	[Admin/Librarian]	Update or delete
GET/POST	/categories	[Mixed]	List or create categories
GET/POST	/publishers	[Mixed]	List or create publishers

Users

GET	/users	[Admin]	List all members with search + role filter
POST	/users	[Admin]	Create a new member account
GET	/users/:id	[Admin]	Get member profile
PUT	/users/:id	[Admin]	Update member details
DELETE	/users/:id	[Admin]	Deactivate member
GET	/users/:id/borrows	[Admin]	Member's full borrow history

Borrowing

GET	/borrow	[Any]	List borrow records (filter by status)
POST	/borrow	[Admin/Librarian]	Issue a book to a member

POST	/borrow/:id/return	[Admin/Librarian]	Record a return. Decrements fine if overdue.
POST	/borrow/:id/renew	[Any]	Extend due date by 14 days (max 2 renewals)

Reservations

GET	/reservations	[Any]	List reservations
POST	/reservations	[Any]	Create a hold request
DELETE	/reservations/:id	[Admin/Librarian]	Cancel a reservation

Fines

GET	/fines	[Any]	List fines (filter by status)
POST	/fines/:id/pay	[Admin/Librarian]	Mark a fine as paid

Notifications

GET	/notifications	[Any]	List notifications for current user
POST	/notifications/:id/read	[Any]	Mark a notification as read
POST	/notifications/read-all	[Any]	Mark all notifications read

Dashboard & Analytics

GET	/dashboard/stats	[Any]	Counts: totalBooks, totalUsers, activeBorrows, overdueBooks, totalFines
GET	/dashboard/overdue	[Any]	List overdue loans
GET	/dashboard/popular-books	[Any]	Top borrowed books
GET	/dashboard/recent-activity	[Any]	Latest activity log entries
GET	/dashboard/borrow-trend	[Any]	Daily borrow counts for last 7 days
GET	/dashboard/category-dist	[Any]	Category distribution for charts

Activity Log

GET	/activity	[Admin]	Full paginated audit log
------------	-----------	---------	--------------------------

5. User Roles & Permissions

SmartLib Pro enforces role-based access at two layers: the Express API (route middleware) and the React UI (ProtectedRoute component + sidebar filtering). There are three roles:

A Admin

Can:

- Everything a Librarian can do
- Create, edit, and deactivate any user account
- Delete books from the catalogue
- View the full activity audit log
- Access all system setup screens (authors, categories, publishers)

L Librarian

Can:

- Add and edit books (not delete)
- Issue and process returns/renewals for any member
- Mark fines as paid
- Cancel reservations
- Manage authors, categories, and publishers
- View all borrowing records, reservations, and fines

Cannot:

- Create/edit/delete users
- View activity log

S Student

Can:

- Browse the books catalogue (read-only)
- View their own borrowing history
- View their own reservations
- View their own fines
- View their own notifications

Cannot:

- Manage any records
- View other members' data
- Access admin/setup pages

Permission Matrix

Feature	Admin	Librarian	Student
View book catalogue	'	'	'
Add / edit books	'	'	'
Delete books	'	'	'
Issue / return / renew loans	'	'	'
View all borrow records	'	'	Own only
Create / edit users	'	'	'
Manage fines (mark paid)	'	'	View own
Cancel reservations	'	'	Own only
System setup (authors etc.)	'	'	'
Activity log	'	'	'
Notifications	'	'	Own only

6. Feature Walkthrough

Login Page

The entry point for all users. A bookshelf background image provides visual context. The form accepts email and password. On success, the user object is stored in localStorage and the session cookie is set by the API.

- Validates non-empty fields before calling the API
- Shows a toast notification on login failure with the error message
- Automatically redirects to the Dashboard after login
- Redirects to /login if a protected route is accessed while unauthenticated

Dashboard

Gives at-a-glance visibility into library health. Stat cards pull live data from GET /api/dashboard/stats, with skeleton loaders shown during fetch.

- Stat cards: Total Books, Active Members, Active Borrows, Overdue Books
- Borrowing Trends area chart (live data from /dashboard/borrow-trend)
- Category Distribution pie chart (live from /dashboard/category-dist)
- Overdue Books count badge highlights red when > 0

Books Catalog

Full searchable inventory table. All roles can view; only Admin/Librarian see the Add New Book button.

- Live search by title, author, or ISBN (debounced via React Query)
- Availability badge shows X / Y copies remaining, turns red when 0
- Category badge styled with the category's custom colour
- Add New Book button opens a form for Admin/Librarian

Borrowing Records

Central hub for all loan management. Colour-coded status badges make the state of each loan immediately clear.

- Tab filters: All | Active | Overdue | Returned
- Due date column shows 'X days left', 'Due today', or 'X days overdue' in red
- Return button: records return, updates available_copies, creates activity log entry
- Renew button: extends due_date by 14 days; disabled after 2 renewals or if overdue
- Students can only view their own records

Reservations

Manages book hold requests. When a reserved book becomes available, the librarian can fulfil the hold.

- Filter by status: All | Pending | Fulfilled | Cancelled
- Client-side search filters by book title or member name
- Cancel button visible to Admin/Librarian for pending reservations
- Expiry date shown — reservations auto-expire if unclaimed

Fines Management

Tracks financial obligations for overdue books. Summary cards give a financial overview at a glance.

- Summary cards: Outstanding Fines total, Collected total, Total issued
- Filter by status: All | Unpaid | Paid
- Mark Paid button for Admin/Librarian records the payment timestamp
- Amount displayed in USD with two decimal places

Notifications

Per-user alert feed. Unread notifications are highlighted with a blue tint and a dot indicator.

- Types: Overdue Alert, Fine Issued, Reservation Ready, Due Reminder, General
- Dismiss button marks a single notification read
- Mark All Read button batch-dismisses all with a single API call
- Unread count shown in the page subtitle

Members (Admin only)

User management table. Only admins can access this page — Librarians and Students are redirected.

- Search by name or email
- Role badge colour-coded: Admin (purple), Librarian (blue), Student (grey)
- Add Member dialog collects name, email, password, role, phone, student ID
- Active/Inactive status badge for account state

Activity Log (Admin only)

Immutable chronological feed of all system events for compliance and auditing.

- Shows actor name, action type badge, entity type badge, and description
- Colour-coded action icons: blue (borrow), green (return), yellow (renew), purple (user ops)
- Paginated — 100 records per page by default

System Setup (Admin / Librarian)

Three-tab screen for managing the reference data used throughout the system.

- Authors tab: grid of author cards with name/bio, Add Author dialog
- Categories tab: colour pill badges, Add Category dialog with colour picker
- Publishers tab: grid with name/website, Add Publisher dialog

7. Authentication Flow

Login Sequence

- 1 User submits email + password via the Login page form
- 2 React calls POST /api/auth/login via the generated useLogin() mutation hook
- 3 Express looks up the user by email in the users table
- 4 PBKDF2 verification: extracts stored salt, re-hashes the submitted password, compares
- 5 On match: generates a 64-char random session token, stores userId !' token in the in-memory Map
- 6 Sets an httpOnly cookie named session_token (7-day max age) on the response
- 7 Returns the user object (id, email, name, role) — NO password fields ever returned
- 8 Frontend stores user object in localStorage and AuthContext
- 9 Wouter router redirects to /

Subsequent Authenticated Requests

After login, every API call made by the React frontend automatically includes the session cookie because the custom fetch wrapper sets credentials: "include" on all fetch requests. The shared reverse proxy routes /api/* calls to the Express backend, which reads the session_token cookie, looks up the user ID in the sessions Map, and attaches the user to the request for route handlers to use.

Logout

POST /api/auth/logout deletes the token from the sessions Map and clears the cookie. The frontend removes the user from localStorage and AuthContext, then redirects to /login.

Session Persistence Note

& In-Memory Sessions

Sessions are stored in a JavaScript Map inside the API server process. This means all sessions are lost when the server restarts. For production deployments requiring persistent sessions, replace the Map with a Redis store or a database-backed session table.

Password Hashing Detail

SmartLib Pro uses Node.js's built-in crypto.pbkdf2Sync with the following parameters:

Parameter	Value
Algorithm	PBKDF2-SHA512
Iterations	1000
Key length	64 bytes (128 hex chars)
Salt	16 random bytes per password (32 hex chars)
Storage format	salt:hash (separated by colon)

8. Demo Accounts & Quick Start

Seeded Demo Accounts

Role	Name	Email	Password
Admin	Dr. Sarah Mitchell	admin@smartlib.edu	admin123
Librarian	James Rodriguez	librarian@smartlib.edu	librarian123
Student	Alice Chen	alice.chen@student.edu	student123
Student	Bob Johnson	bob.johnson@student.edu	student123
Student	Carol Smith	carol.smith@student.edu	student123
Student	David Lee	david.lee@student.edu	student123

Pre-loaded Seed Data

- 12 books across 8 categories (Fiction, Science, History, Technology, Mathematics, Literature, Biography, Philosophy)
- 8 authors (Orwell, Rowling, Hawking, Harari, Sagan, Huxley, Gladwell, Feynman)
- 5 publishers (Penguin Random House, HarperCollins, OUP, MIT Press, Cambridge)
- 8 borrow records — 2 overdue, 4 active, 2 returned
- 3 fines — 2 unpaid (Alice: \$3.50 total), 1 paid (David: \$8.00)
- 4 reservations — 3 pending, 1 fulfilled
- 6 notifications across different types
- 8 activity log entries

Quick Start

Steps

- 1 Open the app in the preview pane (it loads the Login page).
- 2 Enter admin@smartlib.edu and password admin123, then click Sign In.
- 3 You are taken to the Dashboard — review the live stats cards and charts.
- 4 Click Books Catalog in the sidebar to browse the 12 pre-loaded books.
- 5 Click Borrowing in the sidebar to see the 8 borrow records. Notice the 2 overdue rows in red.
- 6 Click Fines to see the outstanding \$3.50 owed by Alice. Use Mark Paid to clear one.
- 7 Click Notifications to see the 6 seeded alerts. Dismiss one or mark all read.
- 8 Click Members (admin-only) to see the 6 user accounts. Add a new student account.
- 9 Click Activity Log to see the full audit trail of seeded events.
- 10 Log out, log back in as alice.chen@student.edu (password: student123). Notice the restricted sidebar.

9. Deployment & Configuration

Development Commands

Command	Purpose
<code>pnpm --filter @workspace/api-server run dev</code>	Start API server (builds then runs on port 8080)
<code>pnpm --filter @workspace/smartlib-pro run dev</code>	Start React dev server (port 23393 with HMR)
<code>pnpm run typecheck</code>	Full end-to-end TypeScript check across all packages
<code>pnpm --filter @workspace/api-spec run codegen</code>	Regenerate API hooks + Zod schemas from OpenAPI
<code>pnpm --filter @workspace/db run push</code>	Push Drizzle schema to the database (dev only)

Environment Variables

Variable	Required	Description
<code>DATABASE_URL</code>	Yes	PostgreSQL connection string (Replit provides this automatically)
<code>SESSION_SECRET</code>	Yes	Secret used for session signing (set in Replit Secrets)
<code>PORT</code>	Yes (auto)	Service port — injected by the Replit workflow runner
<code>BASE_PATH</code>	Yes (auto)	URL prefix for the frontend — injected by Replit artifact routing

Publishing to Production (Replit)

- Click the Deploy button in the Replit workspace (or use the `suggest_deploy` tool).
- Replit builds and bundles both the API server and the React frontend.
- The app becomes available on a `.replit.app` subdomain over HTTPS.
- Production `DATABASE_URL` is automatically provisioned — separate from the dev database.
- To apply schema changes in production: run `pnpm --filter @workspace/db run push` with the production `DATABASE_URL`.

Scaling Considerations

Session Storage

The current in-memory session store works for a single server instance. For horizontal scaling or restarts, replace it with a Redis-backed or PostgreSQL-backed session store.

Fine Calculation

Fines are currently created manually by librarians. To automate, add a scheduled job (e.g. `node-cron`) that queries overdue borrow records nightly and inserts fine rows.

File Uploads

Book cover images are currently stored as URLs (coverUrl field). To support actual file uploads, integrate Replit Object Storage or an external CDN.

Project File Pointers

File / Directory	Purpose
lib/api-spec/openapi.yaml	Single source of truth for the API contract
lib/db/src/schema/	One file per Drizzle table definition
artifacts/api-server/src/routes/	One Express router file per feature area
artifacts/api-server/src/lib/auth.ts	PBKDF2 hash + verify + session token helpers
artifacts/smartlib-pro/src/pages/	One React component file per page
artifacts/smartlib-pro/src/hooks/use-auth.tsx	Auth context, localStorage persistence
artifacts/smartlib-pro/src/components/layout.tsx	Sidebar + mobile nav + role-filtered menu
lib/api-client-react/src/custom-fetch.ts	Base fetch wrapper (credentials: include)